

M10 · agent mode deep dive

Agent Mode Deep Dive

from paradigm to practice

Yesterday: The Paradigm

M7 showed you **the shift**: the agent runs the bottom of the loop — implement, test, review.

Design and plan stay **yours**.

Today: The Practice

Agent mode in VS Code is how you **drive** that loop.

You'll see it plan. You'll watch it work. You'll learn when to steer and when to let it run.

What Is Agent Mode?

A **workspace-aware execution mode** inside VS Code.

Pick **Plan Mode** when you want a written plan first — no files change. Pick **Agent Mode** when you want it to execute.

In Agent Mode the agent **executes**: edits files, runs terminal commands, reads test output, and loops until complete or blocked.

As of VS Code 1.124 (June 2026) **Autopilot is on by default** — the agent auto-executes its todo list and only pauses for terminal commands. Use Plan Mode (or tighten `chat.permissions.default`) when you want to see the plan first.

You are **steering**, not typing.

Four Core Mechanics

1. Plan Mode

Distinct mode — writes a plan with no side effects. Switch to Agent to execute.

2. File Edits

Multi-file changes without manual switching.

3. Terminal Integration

Runs commands: `terraform validate`, `pwsh`, tests.

4. Self-Correction

Reads error output, proposes fix, re-runs.

The Iteration Loop



You own the steering wheel. The agent owns the typing.

Multi-Step Tasks

Prompt: "Refactor this PowerShell function to use approved verbs and add parameter validation."

AGENT PLAN (5 STEPS)

1. Rename function to approved verb
2. Update verb usage internally
3. Add `[ValidateSet]` attributes
4. Update help comments
5. Run `Invoke-ScriptAnalyzer`

RESULT

- Function renamed
- Validation added
- Analyzer passes
- Help updated

One prompt. Five actions.

When to Interrupt, When to Let It Run

INTERRUPT WHEN:

- Plan is wrong (scope/constraints mismatch)
- Agent edits files outside intended scope
- Proposed change violates team standards

LET IT RUN WHEN:

- Plan looks solid and matches intent
- Agent is self-correcting (error → fix → retry)
- Task is well-scoped with enough context

Think of it like pair programming. If your pair starts refactoring the wrong module, you stop them. If they're debugging a test failure and getting close, you let them finish.

Prompt Patterns That Work

✓ Effective Prompts

- **Task + scope:** "Add error handling to `Invoke-AzResourceQuery` . Only modify the function."
- **Goal + constraints:** "Refactor to `for_each` . Preserve resource names."
- **Multi-file context:** "Update API client to new auth flow. Token structure is in `auth/token.ts` ."

✗ Ineffective Prompts

- **Vague:** "Make this better." (Agent guesses wrong)
- **Ambiguous scope:** "Fix the tests." (Which tests?)
- **No context:** "Update to v2 API." (Where's the spec?)

Model Choice: MAI-Code-1-Flash

TECHNICAL PREVIEW

MAI Coding Models

MAI
Image-2.5



MAI
Image-2.5-Flash



MAI
Transcribe-1.5



MAI
Thinking-1



MAI
Voice-2



MAI
Voice-2-Flash



MAI
Code-1-Flash



- New Microsoft AI model family for **agentic coding**
- **MAI-Code-1-Flash** leads — lightweight, inference-efficient
- Trained directly with the **GitHub Copilot harnesses** used in production — learns the harness, not just abstract benchmarks
- Selectable from the **model picker** in GitHub Copilot + VS Code — no setup
- Beats **Claude Haiku 4.5** on SWE-Bench Verified/Pro/Multilingual + Terminal Bench 2 with up to **60% fewer tokens**
- Available on Free / Student / Pro / Pro+ / Max

Multi-Model: The Rubber Duck Agent

Build FY26 · a built-in critic subagent in Copilot CLI

A constructive critic

Gives a second opinion on plans, code, and tests.

Different model

Runs on a **different model** than the session — Claude session → GPT critic, and vice versa. No shared blind spots.

Categorized feedback

Blocking · **Non-blocking** · **Suggestions**.

Read-only

Never edits — the main agent decides what to do with the feedback. Named after the "rubber ducking" technique.

M10 · agent mode deep dive

Live Demo: Terraform Validation

Task: Add input validation to an existing Terraform variable block.

Prompt: "Add validation to the **environment** variable. Only allow 'dev', 'test', 'prod'. Update tests to cover validation."

Watch the agent:

- Plan the changes (variables.tf + tests)
- Edit both files
- Run **terraform validate**
- Self-correct if validation fails

M10 · agent mode deep dive

Next: You Drive the Loop

In 20 minutes, you'll enter the **Agent Mode Playground** (L3).

Your prompt. Agent's plan. Your steering.

Roadmap: L2 (Prompt Dojo) → **L3 (Agent Mode Playground)** → L4 (Multi-File Refactor) → L5 (IaC Workflow)